QueryAPIGate HOW-TO GUIDE 16 · ACCESS

Restrict a key to specific tables

Current as of QueryAPIGate **v0.49.0** · PDF made 4 November 2026

Latest version: anantharajuc.github.io/QueryAPIGate-website/Guide.dc.html?g=16-restrict-a-key-to-tables

---

Time: 5 minutes. **You'll end up with:** a key that can run SQL on a connection but touch only the tables you name - in ad-hoc SQL and saved queries alike, however the forbidden table is reached (a join, a subquery, a CTE).

A `connections` grant is all or nothing for a connection. `allowed_tables` narrows it: every statement the key runs is parsed, and refused if it touches any table not on the list - read or write.

What you'll need

The `sqlglot` parser: `pip install "queryapigate[flow]"` (included in `[all]` and the Docker image). Without it, a table-restricted key's queries fail with a `500` naming the missing package - they are never let through unchecked.

Step 1: Create the key

shell

```
curl -X POST http://127.0.0.1:5000/api/v1/api-keys -H 'X-API-Key: demo-key' -H 'Content-Type:
application/json' -d '{
  "name": "support-desk", "connections": ["shop"], "allowed_tables": ["orders", "customers"]
}'
```

Step 2: See what it allows

A join between two allowed tables works:

shell

```
curl -X POST http://127.0.0.1:5000/execute_sql -H 'X-API-Key: <support-desk secret>' -H 'Content-Type:
application/json' \
  -d '{"sql": "SELECT o.id, c.name, o.total FROM orders o JOIN customers c ON c.id = o.customer_id",
"connection_name": "shop"}'
# [{"id": 1, "name": "Ann", "total": 42.5}, {"id": 2, "name": "Bo", "total": 87.25}, {"id": 3, "name":
"Ann", "total": 12.0}]
```

Every way of reaching another table is refused with `403` and `"code": "table_not_allowed"`, verified one by one:

STATEMENT	RESULT
`SELECT * FROM salaries`	refused
`SELECT * FROM orders WHERE total > (SELECT AVG(amount) FROM salaries)`	refused - the subquery counts
`WITH s AS (SELECT * FROM salaries) SELECT * FROM s`	refused - the CTE's source counts
`SELECT * FROM main.salaries`	refused - a schema prefix doesn't hide it
`SELECT * FROM archive.orders`	refused - the same name in another schema or attached database is another table
`SELECT * FROM sqlite_master`	refused - system catalogues are tables too
`WITH big AS (SELECT * FROM orders WHERE total > 40) SELECT * FROM big`	allowed - `big` is the CTE's name, not a table
`SELECT * FROM ORDERS`	allowed - names match case-insensitively

The error names both lists:

```json
{"error": "This API key may only query these tables: main.customers, main.orders. Forbidden: main.salaries.",
 "code": "table_not_allowed", "request_id": "ac0ec09aa1d6"}
```

Saved queries are checked the same way, every time they run. `orders_by_status` (reads `orders`) works for this key; `salary_list` (reads `salaries`) is refused with the same error, even though the key may use the `shop` connection.

Writes are checked too: if the key also has `allow_writes`, `UPDATE` / `DELETE` / `INSERT` are allowed only on listed tables.

What else it covers, and where it stops

- **Database types:** supported for PostgreSQL, MySQL, SQLite, DuckDB, ClickHouse and Trino - the dialects the parser handles. On **Trino** the match is **fully qualified** (`catalog.schema.table`): a grant of `orders` means the connection's default catalog and schema only, never `orders` in another catalog or schema - see Connect to Trino. On an `h2` or `jdbc` connection a table-restricted key is **refused on every query** (`403` naming the connection type), never left unrestricted; on `mongo` the list is the collections the key may read. See the support matrix.
- **Schema-qualified** (from 0.43; 0.42 and earlier matched bare names): a bare entry means the table in the connection's own database or schema - MySQL's and ClickHouse's `database`, PostgreSQL's `public` (or the connection's `options.schema`), SQLite's and DuckDB's `main`. So `orders` doesn't reach `archive.orders`; list that as `archive.orders`. Names are matched case-insensitively.
- **Statements that hide their tables are refused** (`403 table_check_failed`): `SHOW`, `SET`, `USE`, `ATTACH`, `PRAGMA`, `CALL`, `PREPARE` / `EXECUTE`, cursors. `EXPLAIN` works, checked against the statement it explains.
- **The schema shows only the listed tables.** The schema browser (`GET /connections/shop/schema`) and MCP's `list_tables` leave the others out, and a foreign key pointing at one shows as none. `table_ddl` for a forbidden table answers `404` - *'salaries' is not a table on 'shop'* - exactly as for a table that doesn't exist.

- **Table functions that read data** - DuckDB's `read_parquet(...)` and `read_csv(...)`, ClickHouse's `url(...)`, Trino's `TABLE(system.query(...))` and `table_changes(...)` - are refused for a table-restricted key (`table_not_allowed`): they name no table the list could check. Pure generators (`range`, `numbers`, `generate_series`, `unnest`) are fine. On DuckDB, give files a name with a view and list the view (guide 41).

`allowed_tables` is defence in depth on top of a database user with only the rights it needs - not a replacement for one.

Changing or removing it

```shell
curl -X PATCH http://127.0.0.1:5000/api/v1/api-keys/support-desk -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' \
  -d '{"allowed_tables": ["orders", "customers", "products"]}'
curl -X PATCH http://127.0.0.1:5000/api/v1/api-keys/support-desk -H 'X-API-Key: demo-key' -H 'Content-Type: application/json' \
  -d '{"allowed_tables": null}'   # no restriction
```

Both apply from the key's next request; the secret doesn't change.

Next steps

- Create a role to give several keys the same table list.
- Allow a saved query to write data - `allowed_write_ops` narrows the *kind* of write the same way this narrows the tables.
- Read this project's threat model - what the SQL checks do and don't defend against.